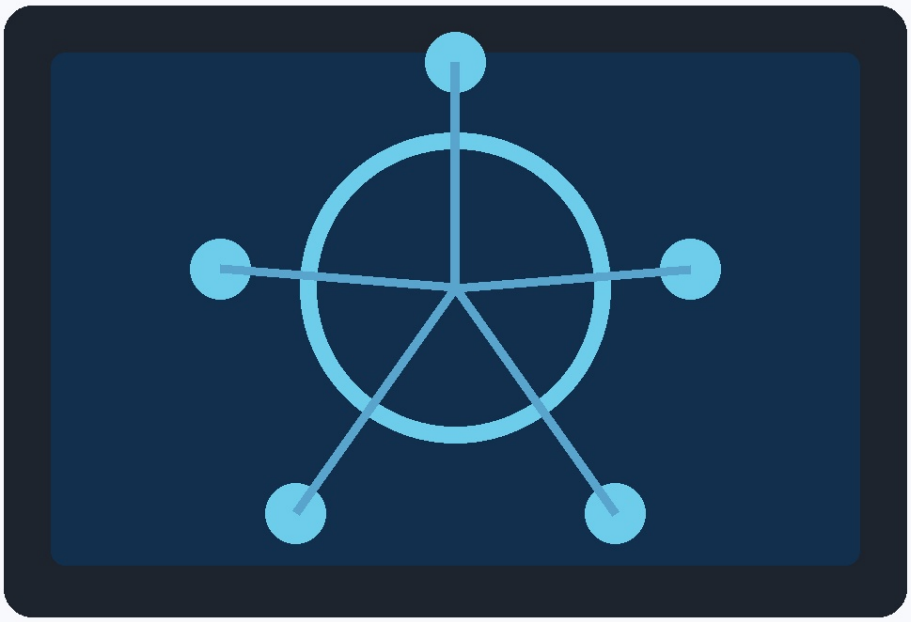
NEXASTORE

Enterprise Knowledge Library



LOCAL AI

Local AI Environment Troubleshooting Guide

Diagnose runtimes, GPU acceleration, models, memory and RAG stacks

NexaStore Knowledge Series • 2026 Edition

1. Diagnose the Layer, Not Just the Error

A local AI application can fail at the hardware, driver, runtime, model, application, vector-store or network layer. Isolating the layer prevents random reinstallations and helps a support agent retrieve the right knowledge.

Symptom	Likely layer	First checks
GPU not visible	Driver/runtime	OS device status, supported driver, runtime GPU detection
Model fails to load	Model/memory	Model format, RAM/VRAM, quantization, disk space
Out of memory	Hardware/runtime	VRAM/RAM usage, context, batch, competing processes
Very slow generation	Acceleration/model	GPU offload, model size, thermal state, token context
RAG returns no documents	Retrieval	Ingestion status, embedding model, index, metadata filters
RAG retrieves irrelevant chunks	Retrieval quality	Chunking, query, metadata, hybrid search/reranking
Tool/agent call fails	Application/integration	Endpoint, auth, schema, permissions, timeout

2. Local Model Runtime

Check	Why
Runtime service is running	Applications cannot call a stopped local inference server
Correct model exists locally	Model name/tag mismatch is common
GPU acceleration active	CPU fallback can appear as severe slowness
VRAM is sufficient	Large models/context can exceed available memory
Port is reachable	Local firewall, binding or container networking can block calls

3. RAG Troubleshooting

Pipeline stage	Failure example	Diagnostic
Ingestion	PDF never indexed	Check document-processing logs and status
Chunking	Answers miss context	Inspect chunk boundaries and overlap
Embedding	Search finds nothing	Verify same embedding space/model for documents and query
Vector index	Expected record absent	Check index count and metadata
Retrieval	Wrong product/policy	Inspect filters, top-k and query rewriting
Reranking	Weak result promoted	Inspect candidate set and reranker scores
Generation	Answer contradicts source	Enforce grounded prompt/citations and tool-vs-RAG boundaries

4. Containers and Local AI

• Confirm GPU passthrough/runtime support if inference runs inside a container.

• Do not assume localhost inside a container means the host machine.

• Verify model volume mounts and permissions.

• Check memory limits applied to the container.

• For multi-service RAG stacks, test inference, embedding, vector database and API services independently.

5. Agentic System Failures

Failure	Check
Agent loops repeatedly	Termination rules, tool errors, state transitions and max iterations
Wrong agent handles request	Routing intent, supervisor prompt and capability registry
Agent calls unauthorized tool	Identity, RBAC, MCP/tool allow-list and policy enforcement
Tool schema error	Function/MCP contract and serialized arguments
Hallucinated action	Require tool result before claiming transactional success
Stale policy answer	Index freshness, metadata/version filtering and source citation

6. Support Rule

Troubleshooting knowledge can come from RAG, but live runtime state should come from diagnostics or authorized tools. An LLM should never claim that a model loaded, a GPU is active, an index contains documents, or an order action succeeded unless a tool confirms it.

Educational notice

This fictional guide supports the NexaStore course case study and uses technology-agnostic diagnostic principles.